

UNIVERSIDADE ESTADUAL DO RIO GRANDE DO NORTE
CAMPUS AVANÇADO NATAL
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO
NCC0235 - COMPUTAÇÃO GRÁFICA

ISMAEL DOS SANTOS MENEZES
MICAEL BRUNO CASSIANO SOARES

ESTUDO DA TÉCNICA DE RAY TRACING

NATAL/RN
2026

ISMAEL DOS SANTOS MENEZES
MICAEL BRUNO CASSIANO SOARES

ESTUDO DA TÉCNICA DE RAY TRACING

Trabalho acadêmico apresentado à disciplina NCC0235 - Computação Gráfica - do curso de Graduação em Ciência da Computação da Universidade Estadual do Rio Grande do Norte, como requisito parcial para avaliação da referida disciplina.

Orientadora: Profa. Dra. Adriana Takahashi

NATAL/RN
2026

RESUMO

Este trabalho descreve a implementação, em C# sobre a plataforma .NET, de um sistema de renderização por *ray tracing* com interface Windows Forms, desenvolvido com o propósito de integrar, de forma coerente e verificável, os principais conceitos de Computação Gráfica. O motor, organizado em camadas — álgebra vetorial, geometria, cena/sombreamento e renderização —, implementa a geração de raios primários a partir de uma câmera *pinhole* com campo de visão ajustável; as interseções raio-esfera, pela equação quadrática na forma *half-b*, e raio-triângulo, pelo algoritmo de Möller–Trumbore; o Modelo de Refletância de Phong, com os termos ambiente, difuso e especular; sombras por raios de sombra, com *offset* numérico que evita auto-interseção; e reflexão recursiva com controle de profundidade. O laço de renderização paraleliza o processamento por linha e suporta sobreamostragem para anti-*aliasing*; a conversão do *buffer* de cores em imagem utiliza Bitmap.LockBits, e a exportação é feita em PNG. A interface permite ajustar resolução (entre 5×55×5 e 1280×7201280×720), posição da câmera, campo de visão e número de amostras por pixel, exibindo o resultado a cada renderização. A cena de demonstração reúne duas esferas e três triângulos, quatro materiais com coeficientes contrastantes e duas fontes de luz, exercitando todos os componentes simultaneamente. A implementação foi verificada com exemplos numéricos calculados manualmente, que confirmaram a correspondência entre a formulação teórica e o comportamento observado. Os resultados evidenciam a articulação entre geometria, iluminação e visualização na síntese de imagens, bem como a importância da robustez numérica e da separação de responsabilidades no projeto de um renderizador.

Palavras-chave: ray tracing; modelo de Phong; interseção raio-triângulo; Möller–Trumbore; Windows Forms.

SUMÁRIO

1. INTRODUÇÃO 4

1.1. CONTEXTUALIZAÇÃO 4

1.2. OBJETIVOS 5

2. ARQUITETURA DO SISTEMA 7

2.1. VISÃO GERAL DA ARQUITETURA 7

2.3. FLUXO DE EXECUÇÃO 10

2.3.1. Fluxo de uma renderização 11

2.3.2. Fluxo de um raio 12

3. IMPLEMENTAÇÃO 14

3.1. ÁLGEBRA VETORIAL 14

3.2. GERADOR DE RAIOS 15

3.2.1. Construção da base ortonormal 16

3.2.2. Plano de imagem 16

3.2.3. Geração do raio por pixel 17

3.2.4. Verificação com grade 3×3 17

3.3. INTERSEÇÃO RAIO-ESFERA 18

3.3.1. Verificação numérica 19

3.4. INTERSEÇÃO RAIO-TRIÂNGULO 19

3.4.1. Verificação numérica 21

3.5. MODELO DE PHONG 21

3.5.1. Verificação numérica 23

3.6. SOMBRAS 23

3.7. REFLEXÃO RECURSIVA 24

3.8. LOOP DE RENDERIZAÇÃO 26

3.9. CONVERSÃO E EXPORTAÇÃO 27

4. CENÁRIO DE DEMONSTRAÇÃO 30

4.1. DESCRIÇÃO DA CENA 30
4.2. JUSTIFICATIVA DAS ESCOLHAS 32
5. CONCLUSÃO 35

1. INTRODUÇÃO

1.1. CONTEXTUALIZAÇÃO

A Computação Gráfica ocupa-se da síntese de imagens por meio de computadores, articulando modelos geométricos, modelos físicos de interação entre luz e matéria e modelos perceptuais. No centro dessa articulação está o problema da **renderização**: dada uma cena tridimensional — objetos, materiais, fontes de luz e uma câmera virtual —, produzir uma imagem bidimensional que represente essa cena de forma coerente com a percepção humana.

Duas famílias de técnicas dominam esse problema. A **rasterização**, adotada pela maioria dos pipelines interativos, projeta os objetos da cena sobre o plano de imagem e preenche os pixels cobertos por cada primitiva, aplicando sombreamento local. É eficiente e adequada à renderização em tempo real, mas trata a iluminação de forma aproximada: reflexos, refrações e sombras exigem técnicas auxiliares (mapas de sombra, mapas de ambiente, *render targets*) que simulam efeitos sem reproduzir seu comportamento físico. A segunda família, o ***ray tracing***, inverte a lógica: em vez de projetar objetos sobre a imagem, traça raios a partir da câmera através de cada pixel e determina o que cada raio encontra. Essa inversão permite calcular reflexos, refrações e sombras de forma direta e conceitualmente simples, ao custo de maior esforço computacional.

O *ray tracing* fundamenta-se no modelo de óptica geométrica: a luz propaga-se em linha reta, e o que o observador vê em cada direção é a luz que chega de trás para frente ao longo dessa linha. Como traçar raios das fontes para a câmera seria proibitivo — a imensa maioria dos raios nunca atinge o observador —, o algoritmo opera no sentido inverso, partindo da câmera em direção à cena. Cada raio primário é testado contra os objetos; no ponto de interseção mais próximo, calcula-se a cor por meio de um modelo de iluminação local, complementado por raios secundários que capturam sombras e reflexões. O resultado é uma imagem que incorpora, de forma integrada, os efeitos que a rasterização trata separadamente.

A qualidade visual do *ray tracing* depende de três pilares interdependentes: a **geometria** (que determina o que cada raio atinge), a **iluminação** (que determina como a luz interage com as superfícies) e a **câmera** (que determina o que é visto e como). O modelo de refletância de Phong, proposto por Bùi Tường Phong em 1975,

fornece uma aproximação empírica e computacionalmente barata para a iluminação local, combinando componentes ambiente, difusa e especular. Os testes de interseção raio-esfera e raio-triângulo — este último na formulação de Möller e Trumbore (1997) — fornecem os mecanismos geométricos que conectam cada raio aos objetos da cena. A câmera virtual, por sua vez, traduz a geometria da projeção em perspectiva na geração sistemática de raios primários.

Este trabalho parte desses fundamentos para implementar, do zero, um sistema de *ray tracing* em C# sobre a plataforma .NET, com interface Windows Forms. A opção por implementar o motor sem recorrer a bibliotecas de renderização deve-se ao valor didático: cada equação, cada decisão numérica e cada estrutura de dados tornam-se explícitas, permitindo verificar a correspondência entre a formulação teórica e o comportamento observado. A interface gráfica, por sua vez, cumpre o papel de tornar o sistema utilizável: permite alterar a câmera, o campo de visão, a resolução e a qualidade da amostragem, exibindo o resultado a cada renderização e exportando-o em PNG. O sistema cobre, assim, o ciclo completo da Computação Gráfica aplicada: modelagem da cena, transformações de visualização, projeção, interseção, sombreamento e síntese da imagem.

1.2. OBJETIVOS

O objetivo geral deste trabalho é **implementar um sistema de renderização por *ray tracing* que integre, de forma coerente e verificável, os principais conceitos de Computação Gráfica estudados na disciplina**, disponibilizando uma interface gráfica que permita ao usuário configurar a cena, ajustar parâmetros de visualização e exportar as imagens produzidas.

Esse objetivo geral desdobra-se nos seguintes objetivos específicos:

1. **Implementar a álgebra vetorial** necessária ao motor, com as operações de soma, subtração, produto escalar, produto vetorial e normalização, adotando a convenção de direções unitárias para que o parâmetro *t* dos raios represente distância geométrica real.
2. **Implementar o gerador de raios** a partir de uma câmera virtual *pinhole*, construindo a base ortonormal da câmera, o plano de imagem e a fórmula de geração de raios primários por pixel, com suporte a *jitter* para anti-*aliasing*.

3. **Implementar a interseção raio-esfera** por meio da equação quadrática na forma *half-b*, com tratamento correto dos três casos de raízes (ambas negativas, uma positiva e uma negativa, ambas positivas).
4. **Implementar a interseção raio-triângulo** pelo algoritmo de Möller–Trumbore, com verificação das coordenadas baricêntricas e normal *double-sided*.
5. **Implementar o Modelo de Refletância de Phong**, com os termos ambiente, difuso e especular, tratando corretamente os casos em que a luz está atrás da superfície ou o observador está fora do lobo especular.
6. **Implementar sombras** por meio de raios de sombra, com *offset* numérico que evite auto-interseção.
7. **Implementar reflexão recursiva**, com controle de profundidade, como mecanismo de captura de iluminação global entre superfícies reflexivas.
8. **Organizar o motor em arquitetura modular**, separando álgebra, geometria, cena/sombreamento e renderização, de modo que cada componente possa ser compreendido, testado e substituído isoladamente.
9. **Construir uma interface Windows Forms** que permita ajustar resolução (entre 5 x 5 e 1280 x 720), posição da câmera, campo de visão e número de amostras por pixel, exibindo o resultado a cada renderização e exportando-o em PNG.
10. **Verificar a implementação** por meio de exemplos numéricos calculados manualmente — a grade 3 x 3 para a câmera, a esfera com t = 4 e o triângulo no vértice V_0 —, comparando os resultados do código com os valores esperados.
11. **Compor um cenário de demonstração** que exercite todos os componentes implementados: esferas e triângulos (ambas as primitivas), materiais com diferentes coeficientes de Phong (fosco, polido, reflexivo), múltiplas fontes de luz (principal e preenchimento) e câmera com parâmetros configuráveis.

O conjunto desses objetivos específicos cobre o escopo declarado: a implementação não busca competir com renderizadores de produção em desempenho ou fotorrealismo, mas demonstrar, de forma completa e verificável, como os conceitos fundamentais de Computação Gráfica se articulam em um sistema funcional.

2. ARQUITETURA DO SISTEMA

Esta seção descreve a organização estrutural do sistema, com foco na separação de responsabilidades entre o motor de *ray tracing* e a interface gráfica, na distribuição dos componentes em arquivos e no fluxo de execução desde a interação do usuário até a produção da imagem final.

2.1. VISÃO GERAL DA ARQUITETURA

O sistema foi organizado segundo uma separação estrita entre **motor de renderização** e **interface de usuário**. O motor reside na pasta Core e não referencia nenhum tipo do Windows Forms; a interface reside no formulário Form1 e consome o motor por meio de suas classes públicas. Essa separação produz três benefícios concretos:

1. **Reutilização e portabilidade:** o motor pode ser testado, reutilizado ou portado para outra interface (console, WPF, ASP.NET) sem alteração;
2. **Isolamento de complexidade:** a complexidade da renderização fica isolada dos detalhes de apresentação;
3. **Evolução independente:** a evolução de cada lado ocorre de forma independente.

O motor é composto por treze tipos, organizados em quatro camadas conceituais:

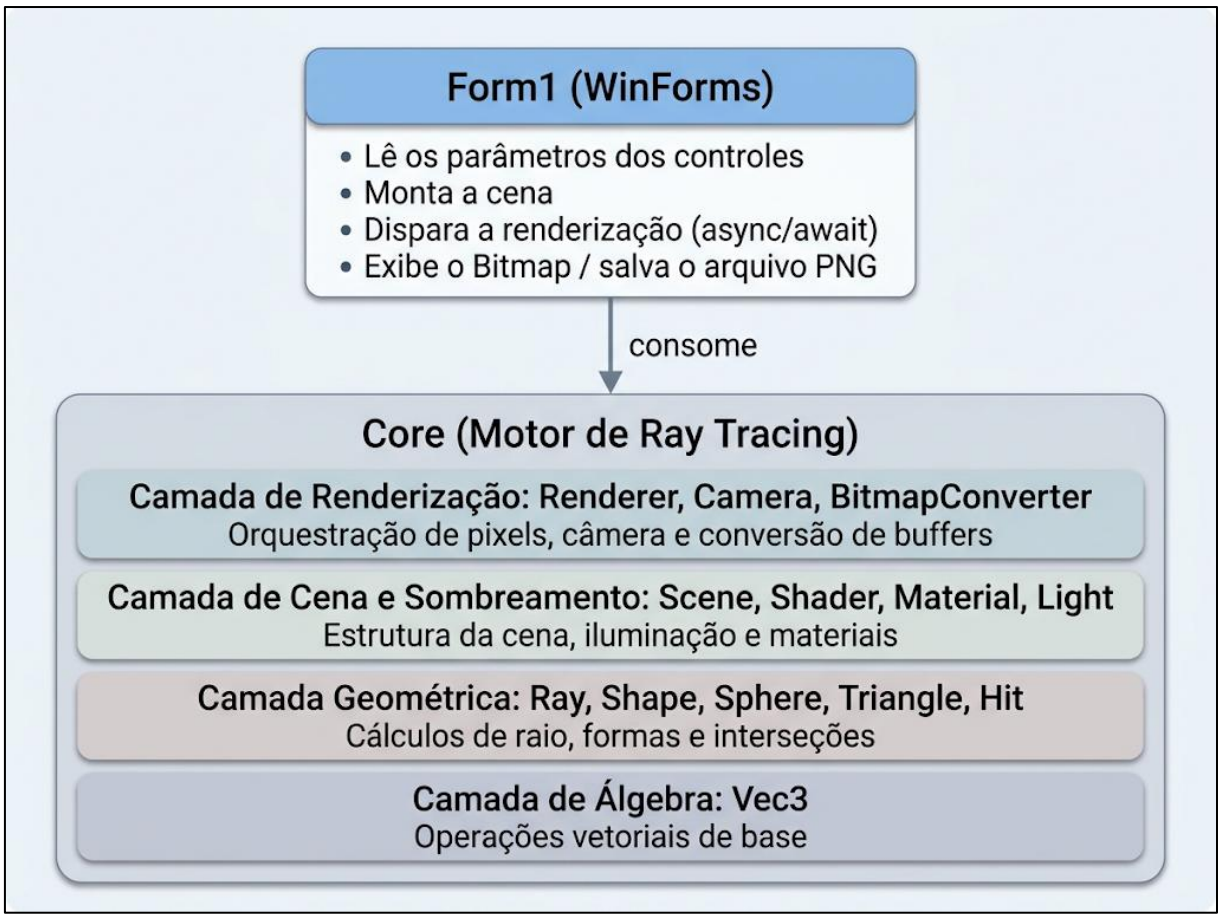
- **Camada de álgebra:** Vec3 fornece as operações vetoriais que sustentam todas as demais camadas. É o único tipo do motor que não depende de nenhum outro.
- **Camada geométrica:** Ray representa o raio paramétrico; Shape é a classe abstrata base; Sphere e Triangle são as duas primitivas concretas. Hit é a estrutura de retorno comum às interseções, carregando distância, ponto, normal e material. Essa camada depende apenas da álgebra.
- **Camada de cena e sombreamento:** Material agrega os coeficientes de Phong e a refletividade; Light representa uma fonte pontual; Scene mantém as listas de objetos e luzes, além de oferecer os métodos Trace (busca da interseção

mais próxima) e InShadow (teste de visibilidade); Shader aplica o modelo de Phong. Essa camada depende da camada geométrica.

- **Camada de renderização:** Camera gera raios primários; Renderer orquestra o laço de pixels e a recursão de reflexão; BitmapConverter traduz o *buffer* de cores em imagem. Essa camada depende de todas as anteriores.

O formulário Form1 atua como **orquestrador**: lê os parâmetros dos controles, monta a cena, instancia a câmera e o renderizador, dispara a renderização em segundo plano e exibe o resultado. Ele não implementa nenhuma lógica de *ray tracing*; apenas conecta a interface ao motor.

A figura a seguir sintetiza a arquitetura em camadas:



A direção das dependências é sempre descendente: a camada de renderização conhece a de cena, que conhece a geométrica, que conhece a álgebra. Nenhuma camada inferior referencia uma superior, o que elimina ciclos e facilita a substituição de componentes.

2.2. Estrutura de pastas e arquivos

O projeto é organizado em uma única solução Visual Studio, com dois namespaces: RayTracerWinForms (interface) e RayTracerWinForms.Core (motor). A pasta Core concentra todos os arquivos do motor; a raiz do projeto contém o formulário e o ponto de entrada.

A tabela abaixo relaciona cada arquivo à sua responsabilidade e à camada conceitual a que pertence.

Arquivo	Camada	Responsabilidade
Core/Vec3.cs	Álgebra	Vetor 3D, operadores, produtos escalar e vetorial, normalização
Core/Ray.cs	Geométrica	Raio paramétrico P(t) = O + tD
Core/Hit.cs	Geométrica	Estrutura de retorno das interseções (t, ponto, normal, material)
Core/Shape.cs	Geométrica	Classe abstrata base das primitivas
Core/Sphere.cs	Geométrica	Interseção raio-esfera (equação quadrática *half-b*)
Core/Triangle.cs	Geométrica	Interseção raio-triângulo (Möller–Trumbore)
Core/Material.cs	Cena/Shading	Coeficientes k_a, k_d, k_s, n e refletividade
Core/Light.cs	Cena/Shading	Fonte pontual (posição, intensidade RGB)
Core/Scene.cs	Cena/Shading	Listas de objetos e luzes; *Trace* e *InShadow*
Core/Shader.cs	Cena/Shading	Modelo de Phong com sombras

Arquivo	Camada	Responsabilidade
Core/Camera.cs	Renderização	Base ortonormal, plano de imagem, geração de raios
Core/Renderer.cs	Renderização	Laço paralelo de pixels e recursão de reflexão
Core/BitmapConverter.cs	Renderização	Conversão do *buffer* de cores em *Bitmap*
Form1.cs	Interface	Lógica do formulário: eventos, montagem da cena, chamada ao motor
Form1.Designer.cs	Interface	Declaração dos controles e do *InitializeComponent*
Program.cs	Interface	Ponto de entrada da aplicação

A separação em arquivos segue o princípio da **responsabilidade única**: cada arquivo contém um único tipo público, com exceção de Shape, que declara apenas a classe abstrata, e dos arquivos parciais do formulário, que são gerados pelo designer. Essa granularidade facilita a navegação, a leitura e a substituição de componentes individuais — por exemplo, substituir a equação quadrática da esfera por uma versão com transformação afim não afeta os demais arquivos.

A pasta Core não contém referências a System.Windows.Forms nem a System.Drawing, exceto por BitmapConverter, que naturalmente depende de System.Drawing para produzir a imagem. Mesmo essa dependência é isolada: o restante do motor trabalha apenas com Vec3 e estruturas próprias, de modo que a renderização pura (sem conversão para imagem) poderia ser executada em qualquer ambiente .NET.

2.3. FLUXO DE EXECUÇÃO

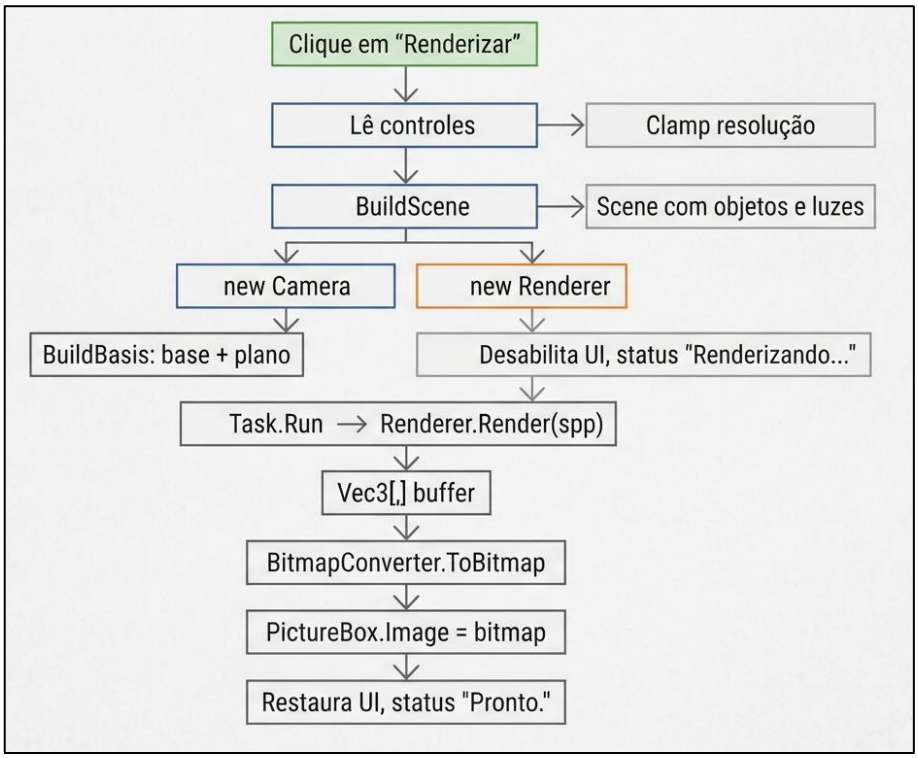
O fluxo de execução do sistema pode ser descrito em duas escalas: o **fluxo de uma renderização** (do clique do usuário à exibição da imagem) e o **fluxo de um raio** (da geração à cor final). Ambos são detalhados a seguir.

2.3.1. Fluxo de uma renderização

O ciclo completo, desde a interação do usuário até a atualização da interface, segue as etapas abaixo:

1. **Leitura dos parâmetros:** O manipulador btnRender_Click lê os valores dos controles: resolução (largura e altura), posição da câmera, campo de visão, número de amostras por pixel. A resolução é limitada ao intervalo [5, 1280] x [5, 720] antes do uso, em conformidade com o requisito do projeto.
2. **Montagem da cena:** O método estático BuildScene constrói uma nova instância de Scene, populando-a com os objetos da demonstração (duas esferas, três triângulos) e as duas fontes de luz. Cada objeto recebe um Material com seus coeficientes.
3. **Construção da câmera e do renderizador:** Com os parâmetros lidos, instancia-se Camera (que já calcula a base ortonormal e as dimensões do plano de imagem no construtor) e Renderer, associando-o à cena e à câmera.
4. **Desabilitação da interface:** O método SetUiEnabled(false) desabilita botões e controles para impedir cliques concorrentes durante a renderização. O rótulo de status muda para "Renderizando..." e o cursor assume a forma de espera.
5. **Renderização em segundo plano:** A chamada await Task.Run(() => renderer.Render(spp)) executa o laço de pixels em uma *thread* do *thread pool*, liberando a *thread* da interface para permanecer responsiva. O resultado é a matriz de cores produzida pelo Renderer.
6. **Conversão para imagem:** O *buffer* é passado a BitmapConverter.ToBitmap, que devolve um Bitmap no formato Format24bppRgb.
7. **Atualização da interface:** O Bitmap novo é atribuído ao PictureBox; a imagem anterior é descartada com Dispose para liberar memória. O rótulo de status exibe um resumo (resolução, FOV, spp) e o título da janela é atualizado.
8. **Restauração da interface:** O bloco finally garante que SetUiEnabled(true) e o cursor padrão sejam restaurados mesmo em caso de exceção, que é capturada e apresentada ao usuário em uma caixa de mensagem.

O diagrama a seguir sintetiza o fluxo:



A exportação em PNG segue um fluxo mais curto, disparado por btnSavePng_Click: verifica-se a existência de _currentBitmap, abre-se um SaveFileDialog, e o Bitmap é gravado por Save(path, ImageFormat.Png). Não há re-renderização; exporta-se exatamente a imagem atualmente exibida.

2.3.2. Fluxo de um raio

Dentro do Renderer, cada raio percorre uma cadeia de operações que combina geração, interseção, sombreamento e recursão.

1. **Geração:** Camera.GetRay(i, j, jx, jy) produz o raio primário com origem na câmera e direção calculada a partir do pixel e do *jitter*.
2. **Busca da interseção mais próxima:** Scene.Trace percorre a lista de objetos; cada objeto chama seu Intersect, que retorna null ou um Hit. O menor *t* válido é mantido. Se nenhum hit ocorrer, retorna-se a cor de fundo.
3. **Sombreamento local:** Shader.ShadePhong acumula o termo ambiente e, para cada luz, verifica visibilidade (InShadow), calcula a componente difusa e a especular, e soma à cor.

4. **Teste de sombra:** Para cada luz, Scene.InShadow lança um raio de sombra da superfície até a fonte; se houver obstrução, a contribuição daquela luz é ignorada.
5. **Reflexão recursiva:** Se o material tem refletividade positiva e a profundidade é menor que MaxDepth, constrói-se o raio refletido, que é rastreado recursivamente. A cor retornada é ponderada por *k_r* e somada.
6. **Clamp:** A cor final é truncada para o intervalo [0, 1] e devolvida ao laço de pixels.

Esse fluxo é repetido para cada amostra de cada pixel. A recursão de reflexão é a única fonte de não linearidade: um pixel cujo raio atinge um material reflexivo dispara uma nova cadeia completa de geração–interseção–sombreamento, até a profundidade máxima.

A separação entre o fluxo de renderização (que orquestra pixels e interface) e o fluxo de um raio (que orquestra geometria, sombreamento e recursão) reflete a divisão de responsabilidades entre Renderer e Scene/Shader: o primeiro decide *quais* raios lançar e *como* combiná-los; os segundos decidem *o que* cada raio encontra e *qual* cor ele produz.

3. IMPLEMENTAÇÃO

Esta seção descreve a implementação do motor de ray tracing desenvolvido, com foco nas decisões técnicas e na correspondência entre os conceitos teóricos apresentados e o código-fonte produzido. O motor foi implementado em C# sobre a plataforma .NET, organizado na pasta “Core” do projeto, de modo a manter independência em relação à interface gráfica. A apresentação segue a ordem de dependência entre os componentes: álgebra vetorial, geração de raios, interseção raio-esfera e interseção raio-triângulo.

3.1. ÁLGEBRA VETORIAL

Toda a formulação do *ray tracing* — geração de raios, testes de interseção e cálculo de sombreamento — é expressa em termos de operações vetoriais tridimensionais. Por essa razão, o primeiro componente implementado foi a estrutura “Vec3”, que encapsula um vetor (ou ponto, ou cor RGB) em $\mathbb{R}^3$ e concentra as operações algébricas utilizadas pelos demais módulos.

A estrutura armazena três componentes de dupla precisão (double X, Y, Z) e implementa:

- **Operadores aritméticos:** soma, subtração (binária e unária), multiplicação por escalar (nas duas ordens) e divisão por escalar.
- **Produto componente a componente (Hadamard):** utilizado para combinar cores e intensidades luminosas canal a canal.
- **Produto escalar (Dot):** base dos cálculos de cosseno entre vetores.
- **Produto vetorial (Cross):** utilizado na construção da base da câmera e no algoritmo de Möller–Trumbore.
- **Comprimento (Length, LengthSq) e normalização (Normalize).**

A normalização merece destaque. Em todo o motor adota-se a convenção de que a direção de um raio é sempre unitária. Isso garante que o parâmetro t na equação paramétrica $\mathbf{P}(t) = \mathbf{O} + t\mathbf{D}$ represente a distância geométrica real entre a origem e o ponto atingido, o que simplifica a comparação entre interseções candidatas e o cálculo de distâncias em raios de sombra.

A estrutura também expõe as constantes Vec3.Zero e Vec3.One, utilizadas como elementos neutros em acumuladores de cor e em inicializações.

```
public struct Vec3
{
    public double X, Y, Z;

    public static Vec3 operator +(Vec3 a, Vec3 b) => new(a.X + b.X, a.Y +
b.Y, a.Z + b.Z);
    public static Vec3 operator *(Vec3 a, double s) => new(a.X * s, a.Y *
s, a.Z * s);
    public static Vec3 operator *(Vec3 a, Vec3 b) => new(a.X * b.X, a.Y *
b.Y, a.Z * b.Z);

    public double Dot(Vec3 b) => X * b.X + Y * b.Y + Z * b.Z;

    public Vec3 Cross(Vec3 b) => new(
        Y * b.Z - Z * b.Y,
        Z * b.X - X * b.Z,
        X * b.Y - Y * b.X);

    public double Length => Math.Sqrt(LengthSq);
    public Vec3 Normalize() => this / Length;

    public static Vec3 Zero => new(0, 0, 0);
}
```

A simplicidade dessa estrutura é intencional: ela não contém lógica de domínio, apenas álgebra. Toda a semântica geométrica (pontos, direções, normais, cores) é atribuída pelo contexto de uso, o que mantém o componente reutilizável e facilmente verificável.

3.2. GERADOR DE RAIOS

O gerador de raios é implementado pela classe “Camera”, responsável por traduzir a geometria da câmera virtual e as coordenadas de cada pixel da imagem em raios primários no espaço do mundo. A implementação segue o modelo de câmera *pinhole* com projeção em perspectiva, conforme descrito conceitualmente.

3.2.1. Construção da base ortonormal

A câmera é definida por quatro parâmetros: posição do observador (Eye), ponto de interesse (LookAt), vetor de orientação vertical (Up), ângulo de campo de visão vertical (FovDegrees) e resolução da imagem (Width, Height). A partir desses dados, o construtor invoca o método BuildBasis, que constrói uma base ortonormal **(u', v, w)** associada à câmera:

$$\mathbf{w} = (\mathbf{e} - \mathbf{look}) / ||\mathbf{e} - \mathbf{look}||$$
$$\mathbf{u'} = (\mathbf{up} \times \mathbf{w}) / ||\mathbf{up} \times \mathbf{w}||$$
$$\mathbf{v} = \mathbf{w} \times \mathbf{u'}$$

O vetor **w** aponta da cena para a câmera (direção "para trás"); **u'** é o eixo horizontal ("direita") e **v** é o eixo vertical ("cima"). A ortonormalidade é garantida pela normalização de **w** e **u'** e pelo fato de **v** ser o produto vetorial de dois vetores unitários ortogonais.

3.2.2. Plano de imagem

A uma distância fixa $d = 1$ da câmera, o plano de imagem tem meia-altura e meia-largura dadas por:

$$h = \tan(\theta_{fov} / 2) \cdot d$$
$$w = h \cdot (W / H)$$

onde W e H são, respectivamente, a largura e a altura da imagem em pixels. O fator W / H preserva a proporção da imagem (*aspect ratio*), evitando distorção quando a resolução não é quadrada. A meia-altura h é recalculada no construtor, de modo que a alteração do campo de visão ou da resolução exige apenas a reinstanciação da câmera.

3.2.3. Geração do raio por pixel

Para cada pixel (i, j) — com i variando em [0, W) e j em [0, H) — as coordenadas do ponto correspondente no plano de imagem são:

$$x = ((2(i + j_x) / W) - 1) \cdot w$$
$$y = (1 - (2(j + j_y) / H)) \cdot h$$

O deslocamento j_x, $j_y \in [0, 1)$ permite amostrar qualquer ponto interno ao pixel. Quando $j_x = j_y = 0{,}5$, amostra-se o centro geométrico do pixel; quando assumem valores aleatórios, obtém-se *jittered sampling*, base do anti-*aliasing* por sobreamostragem. A direção do raio é então:

$$\mathbf{D} = \text{normalize}(x\mathbf{u}' + y\mathbf{v} - d\mathbf{w})$$

e o raio primário é (O = **e**, **D**). A origem é comum a todos os raios primários (característica do modelo *pinhole*); o que varia é apenas a direção.

```
public Ray GetRay(int i, int j, double jx = 0.5, double jy = 0.5)
{
    double sx = (2.0 * (i + jx) / Width - 1.0) * _halfW;
    double sy = (1.0 - 2.0 * (j + jy) / Height) * _halfH;

    Vec3 dir = (sx * _right + sy * _up - _d * _forward).Normalize();
    return new Ray(Eye, dir);
}
```

3.2.4. Verificação com grade 3×3

A correção da implementação foi verificada com uma grade de 3 × 3 pixels, câmera em (0, 0, 0), alvo em (0, 0, -1), vetor *up* (0, 1, 0) e θ_fov = 90°. Nesse caso, h = w = 1, e as coordenadas dos centros dos nove pixels assumem valores simétricos:

Pixel (i, j)	x	y	Direção normalizada D
(0, 0)	-0,667	+0,667	(-0,4851, 0,4851, -0,7276)
(1, 0)	0	+0,667	(0, 0,5547, -0,8321)
(2, 0)	+0,667	+0,667	(0,4851, 0,4851, -0,7276)
(0, 1)	-0,667	0	(-0,5547, 0, -0,8321)
(1, 1)	0	0	(0, 0, -1)
(2, 1)	+0,667	0	(0,5547, 0, -0,8321)
(0, 2)	-0,667	-0,667	(-0,4851, -0,4851, -0,7276)
(1, 2)	0	-0,667	(0, -0,5547, -0,8321)
(2, 2)	+0,667	-0,667	(0,4851, -0,4851, -0,7276)

O pixel central (1, 1) produz exatamente (0, 0, -1), apontando para o alvo — resultado consistente com a definição do modelo *pinhole*. A simetria dos demais valores confirma a corretude da base e do plano de imagem.

3.3. INTERSEÇÃO RAIO-ESFERA

A interseção raio-esfera é implementada pela classe Sphere, que herda de Shape e implementa o método abstrato Intersect. Cada instância armazena centro (Center), raio (Radius) e material (Material); a normal não é armazenada, pois varia com o ponto de interseção e é calculada sob demanda.

O teste resolve a equação quadrática descrita na Seção 2.3, na forma *half-b*, que evita o fator 2 na fórmula de Bhaskara:

$$b = \mathbf{oc} \cdot \mathbf{D}, \qquad c = \mathbf{oc} \cdot \mathbf{oc} - r^2, \qquad \Delta' = b^2 - c$$

onde **oc** = *O* - *C*. Se Δ' < 0, não há interseção. Caso contrário, as raízes são $t_1 = -b - \sqrt{\Delta'}$ e $t_2 = -b + \sqrt{\Delta'}$.

A escolha da raiz segue a regra: se $t_1 \geq t_\text{min}$, usa-se t_1; caso contrário, tenta-se t_2. Isso trata corretamente três situações:

1. **Ambas as raízes negativas:** esfera atrás do raio, sem interseção válida;
2. **$t_1 > 0$:** o raio entra na esfera em t_1;

3. **$t_1 < 0 < t_2$:** a origem está dentro da esfera e a saída ocorre em t_2.

A verificação de $t > t_max$ rejeita interseções mais distantes que a atual candidata, o que permite ao chamador podar a busca.

A normal no ponto de interseção é obtida por **N** = (*P* - *C*) / *r*, já unitária por construção, pois ||*P* - *C*|| = *r*.

```
public override Hit? Intersect(Ray ray, double tMin, double tMax)
{
    Vec3 oc = ray.Origin - Center;
    double b = oc.Dot(ray.Direction);
    double c = oc.Dot(oc) - Radius * Radius;
    double disc = b * b - c;
    if (disc < 0) return null;

    double sqrtD = Math.Sqrt(disc);
    double t1 = -b - sqrtD;
    double t2 = -b + sqrtD;

    double t = t1;
    if (t < tMin) t = t2;
    if (t < tMin || t > tMax) return null;

    Vec3 p = ray.At(t);
    Vec3 n = (p - Center).Normalize();
    return new Hit(t, p, n, Material);
}
```

3.3.1. Verificação numérica

Considere raio com *O* = (0, 0, 0), **D** = (0, 0, -1) e esfera de centro *C* = (0, 0, -5) e raio *r* = 1. Então **oc** = (0, 0, 5), *b* = -5, *c* = 25 - 1 = 24, Δ' = 25 - 24 = 1. As raízes são $t_1 = 4$ e $t_2 = 6$; ambas positivas, adota-se *t* = 4. O ponto é *P* = (0, 0, -4) e a normal **N** = (0, 0, 1). O resultado é consistente com o cálculo manual e com a intuição geométrica (a esfera está centrada em *z* = -5, com raio 1, de modo que o raio que viaja ao longo de -*z* a atinge em *z* = -4).

3.4. INTERSEÇÃO RAIO-TRIÂNGULO

A interseção raio-triângulo é implementada pela classe Triangle, com o algoritmo de Möller–Trumbore (Seção 2.4). O triângulo é definido por três vértices V_0,

V_1, V_2; o construtor pré-calcula as arestas $\mathbf{e_1} = V_1 - V_0$, $\mathbf{e_2} = V_2 - V_0$ e a normal geométrica $\mathbf{n}$ = normalize($\mathbf{e_1} \times \mathbf{e_2}$). O pré-cálculo é relevante porque, em uma cena com muitos triângulos, o mesmo objeto é testado repetidamente pelos raios.

O método Intersect resolve o sistema baricêntrico $O + t\mathbf{D} = V_0 + u\mathbf{e_1} + v\mathbf{e_2}$ pela regra de Cramer, na formulação vetorial:

$$\mathbf{T} = \boldsymbol{O} - \boldsymbol{V_0}, \qquad \mathbf{P} = \mathbf{D} \times \mathbf{e_2}, \qquad \det = \mathbf{e_1} \cdot \mathbf{P}$$

$$u = (\mathbf{T} \cdot \mathbf{P}) / \det, \qquad \mathbf{Q} = \mathbf{T} \times \mathbf{e_1}, \qquad v = (\mathbf{D} \cdot \mathbf{Q}) / \det, \qquad t = (\mathbf{e_2} \cdot \mathbf{Q}) / \det$$

Os testes de aceitação seguem a formulação: $|\det| < \varepsilon$ indica raio paralelo ao plano do triângulo; $u < 0$, $u > 1$, $v < 0$ ou $u + v > 1$ indicam que o ponto está fora do triângulo; $t < t_\text{min}$ ou $t > t_\text{max}$ descartam interseções fora do intervalo de interesse. Adota-se $\varepsilon = 10^{-8}$.

A normal devolvida é a normal geométrica, invertida quando necessário para que aponte contra o raio incidente. Essa convenção implementa material *double-sided*: o triângulo é visível de ambos os lados, e o sombreamento é calculado corretamente em cada face, sem que o usuário precise duplicar a geometria.

```
public override Hit? Intersect(Ray ray, double tMin, double tMax)
{
    const double EPS = 1e-8;
    Vec3 T = ray.Origin - V0;
    Vec3 P = ray.Direction.Cross(_e2);
    double det = _e1.Dot(P);
    if (Math.Abs(det) < EPS) return null;

    double invDet = 1.0 / det;
    double u = T.Dot(P) * invDet;
    if (u < 0 || u > 1) return null;

    Vec3 Q = T.Cross(_e1);
    double v = ray.Direction.Dot(Q) * invDet;
    if (v < 0 || u + v > 1) return null;

    double t = _e2.Dot(Q) * invDet;
    if (t < tMin || t > tMax) return null;

    Vec3 p = ray.At(t);
    Vec3 n = _normal;
    if (n.Dot(ray.Direction) > 0) n = -n;
    return new Hit(t, p, n, Material);
}
```

3.4.1. Verificação numérica

Considere o triângulo V_0 = (0, 0, -5), V_1 = (1, 0, -5), V_2 = (0, 1, -5) e o raio O = (0, 0, 0), **D** = (0, 0, -1). As arestas são $\mathbf{e_1}$ = (1, 0, 0), $\mathbf{e_2}$ = (0, 1, 0); **T** = (0, 0, 5); **P** = **D** × $\mathbf{e_2}$ = (1, 0, 0); det = 1. Segue u = 0, **Q** = (0, 5, 0), v = 0 e t = 5. O ponto P = (0, 0, -5) coincide com V_0, caso de borda aceito pelas desigualdades não estritas. A normal geométrica é (0, 0, 1), oposta ao raio incidente — correta para o sombreamento.

Um segundo teste com **D** = normalize(0,2, 0,2, -1) produz $u \approx 0{,}9997$ e $v \approx 0{,}9997$. Como $u + v > 1$, o algoritmo rejeita o *hit* — resultado coerente com a análise geométrica, pois o raio passa fora do triângulo. Esse teste evidencia a importância do par $u + v$ como critério de pertinência: nem u nem v isoladamente o capturam.

3.5. MODELO DE PHONG

O sombreamento local é implementado pela classe estática Shader, cujo método ShadePhong aplica o Modelo de Refletância de Phong (Seção 2.2) a um ponto de interseção. A assinatura recebe o Hit (contendo ponto, normal e material), a direção de visão **V** (unitária, do ponto para a câmera) e a cena (para consultar as luzes e verificar sombras). O retorno é uma cor RGB acumulada.

A equação implementada é:

$$\boldsymbol{I = k_a \cdot I_a + \sum [\, k_d \cdot I_\ell \cdot \max(0, \mathbf{N} \cdot \mathbf{L}) + k_s \cdot I_\ell \cdot \max(0, \mathbf{R} \cdot \mathbf{V})^{n} \,]}$$

onde **N** é a normal no ponto, **L** é o vetor unitário do ponto para a fonte de luz ℓ, **R** = 2(**N** · **L**)**N** - **L** é o vetor refletido e n é o expoente especular (*shininess*).

A implementação percorre as luzes da cena e, para cada uma, executa três verificações antes de acumular a contribuição direta:

1. **Luz atrás da superfície:** se **N** · **L** ≤ 0, a fonte está no lado oposto ao da normal e o termo é descartado. Esse teste é necessário porque o cosseno entre **N** e **L** é negativo nessa configuração, e o modelo não admite contribuição negativa.
2. **Ponto na sombra:** a consulta InShadow (Seção 3.6) verifica se há obstrução entre o ponto e a fonte; em caso afirmativo, a contribuição direta daquela luz é ignorada, restando apenas o termo ambiente.

3. **Cálculo do especular:** o vetor **R** é construído e normalizado, e o fator **R** · **V** é elevado a *n*. Se **R** · **V** ≤ 0, o especular é nulo (observador fora do lobo de reflexão).

O termo ambiente é calculated uma única vez, antes do laço, como *k_a* · *I_a* · cor, onde *I_a* é a luz ambiente global da cena (parâmetro Ambient da classe Scene). Optou-se por manter o termo ambiente separado das luzes para deixar explícito que ele não depende de direção nem de sombra.

```
public static Vec3 ShadePhong(Hit hit, Vec3 viewDir, Scene scene)
{
    Material m = hit.Material;
    Vec3 N = hit.Normal;
    Vec3 color = m.Color * (m.Ka * scene.Ambient);

    foreach (var light in scene.Lights)
    {
        Vec3 L = (light.Position - hit.Point).Normalize();
        double ndotl = N.Dot(L);
        if (ndotl <= 0) continue;
        if (scene.InShadow(hit.Point, light.Position)) continue;

        Vec3 diffuse = m.Color * (m.Kd * ndotl) * light.Intensity;

        Vec3 R = (N * (2.0 * ndotl) - L).Normalize();
        double rdotv = R.Dot(viewDir);
        Vec3 specular = Vec3.Zero;
        if (rdotv > 0)
        {
            double factor = Math.Pow(rdotv, m.Shininess);
            specular = new Vec3(1, 1, 1) * (m.Ks * factor) *
light.Intensity;
        }

        color += diffuse + specular;
    }
    return color;
}
```

Três decisões de projeto merecem registro. Primeiro, a cor especular é sempre branca (ou a cor da luz), independentemente da cor do material — escolha consistente com o modelo de Phong original, no qual o *highlight* é atribuído à fonte de luz e não ao material. Segundo, o termo especular não é multiplicado pela cor do material; fazê-lo produziria *highlights* coloridos, comportamento típico de metais e mais próximo de modelos fisicamente baseados, mas incompatível com a formulação clássica de Phong. Terceiro, a cor resultante pode exceder 1 em algum canal (especialmente

quando $k_a + k_d + k_s > 1$); o *clamp* para [0, 1] é aplicado no Renderer (Seção 3.8), não no Shader, para manter o *shader* puro e reutilizável.

3.5.1. Verificação numérica

Considere um ponto na superfície com **N** = (0, 0, 1), luz pontual em (0, 0, 5) (portanto **L** = (0, 0, 1)) e observador em (0, 0, 5) (portanto **V** = (0, 0, 1)). Material: k_a = 0,1, k_d = 0,7, k_s = 0,5, n = 50; intensidades $I_a = I_\ell = 1$.

- **Ambiente:** $0{,}1 \cdot 1 = 0{,}1$.
- **Difusa:** $\mathbf{N} \cdot \mathbf{L} = 1$, logo $0{,}7 \cdot 1 \cdot 1 = 0{,}7$.
- **Especular:** $\mathbf{R} = (0, 0, 1)$, $\mathbf{R} \cdot \mathbf{V} = 1$, logo $0{,}5 \cdot 1 \cdot 1^{50} = 0{,}5$.
- **Total:** $I = 1{,}3$. O valor superior a 1 é esperado e será truncado para 1 no *clamp* final.

Se o observador é deslocado para (5, 0, 5), então $\mathbf{V} \approx (0{,}707, 0, 0{,}707)$ e $\mathbf{R} \cdot \mathbf{V} \approx 0{,}707$. O especular cai para $0{,}5 \cdot 0{,}707^{50} \approx 3 \times 10^{-8}$, praticamente nulo. O total fica em torno de 0,8, dominado pela difusa. Esse teste evidencia o papel do expoente n na concentração do *highlight*: pequenas variações angulares produzem variações drásticas na contribuição especular.

3.6. SOMBRAS

As sombras são implementadas pelo método InShadow da classe Scene. A técnica consiste em lançar um **raio de sombra** do ponto de interseção em direção à fonte de luz e verificar se há alguma geometria no caminho. Em caso afirmativo, o ponto está obstruído e a contribuição direta daquela luz é descartada pelo Shader.
O raio de sombra tem origem no ponto de interseção e direção **L** = normalize($L_{pos} - P$). O ponto de origem é deslocado ao longo da direção em um *offset* $\varepsilon = 10^{-3}$, e o intervalo de busca é limitado a $[\varepsilon, d - \varepsilon]$, onde $d = ||L_{pos} - P||$ é a distância até a fonte. Dois cuidados numéricos são essenciais:

1. **Auto-interseção (*shadow acne*):** O *offset* na origem evita que o próprio objeto que contém o ponto seja detectado como obstrução, o que produziria sombras espúrias.

2. **Limite superior (*d* - ε):** Evita que a fonte de luz — se for representada como geometria — seja detectada como obstrução; mesmo com fontes pontuais, o limite impede que objetos além da fonte interfiram.

O valor $\varepsilon = 10^{-3}$ é maior do que o $\varepsilon = 10^{-4}$ usado no Renderer (Seção 3.8) para o teste principal; essa diferença reflete o fato de que os raios de sombra partem de superfícies e são mais sensíveis a erros de arredondamento.

```
public bool InShadow(Vec3 point, Vec3 lightPos, double eps = 1e-3)
{
    Vec3 dir = lightPos - point;
    double dist = dir.Length;
    if (dist < eps) return false;

    Vec3 dirNorm = dir / dist;
    Ray shadowRay = new(point + dirNorm * eps, dirNorm);

    var h = Trace(shadowRay, eps, dist - eps);
    return h.HasValue;
}
```

O método InShadow retorna true na presença de qualquer obstrução, sem se preocupar com qual objeto obstrui. Esse comportamento é suficiente porque o modelo de Phong com fontes pontuais produz sombras binárias (*hard shadows*): o ponto ou vê a luz, ou não vê. Sombras suaves exigiriam fontes de área e múltiplos raios de sombra por amostragem, o que está fora do escopo desta implementação.

3.7. REFLEXÃO RECURSIVA

A reflexão recursiva é implementada no método Trace da classe Renderer, que combina o sombreamento local de Phong com a contribuição de raios refletidos. Trata-se de uma extensão do modelo de Phong: o termo de reflexão é **somado** à cor local, ponderado pela refletividade do material.

O procedimento é o seguinte. Após calcular a cor local *I_local* via ShadePhong, verifica-se se o material tem refletividade $k_r > 0$ e se a profundidade atual é menor que o limite MaxDepth = 5. Em caso afirmativo, constrói-se o raio refletido com origem em $P + \varepsilon\mathbf{N}$ e direção:

$$\mathbf{R} = \mathbf{D} - 2(\mathbf{D} \cdot \mathbf{N})\mathbf{N}$$

(equivalente a $2(\mathbf{N} \cdot \mathbf{D})\mathbf{N} - \mathbf{D}$, mas com sinal de **D** absorvido). O raio é então rastreado recursivamente, e a cor retornada é multiplicada por *k_r* e somada à cor local. A cor final é truncada para [0, 1] antes de ser devolvida.

O limite MaxDepth = 5 controla dois aspectos:

1. **Custo computacional:** cada reflexão adicional multiplica o número de raios;
2. **Convergência:** em cenas com múltiplos espelhos, a recursão sem limite não terminaria.

O valor 5 é suficiente para capturar reflexos múltiplos visíveis sem custo excessivo. O *offset* $\varepsilon = 10^{-4}$ na origem do raio refletido evita auto-interseção, pelo mesmo motivo que nos raios de sombra.

```
private Vec3 Trace(Ray ray, int depth)
{
    const double EPS = 1e-4;
    var hit = Scene.Trace(ray, EPS, double.PositiveInfinity);
    if (!hit.HasValue)
        return new Vec3(0.02, 0.02, 0.05);

    Vec3 viewDir = (-ray.Direction).Normalize();
    Vec3 color = Shader.ShadePhong(hit.Value, viewDir, Scene);

    Material m = hit.Value.Material;
    if (m.Reflectivity > 0 && depth < MaxDepth)
    {
        Vec3 N = hit.Value.Normal;
        Vec3    R    =    (ray.Direction    -    N    *    (2.0    *
ray.Direction.Dot(N))).Normalize();
        Ray refl = new(hit.Value.Point + N * EPS, R);
        color += Trace(refl, depth + 1) * m.Reflectivity;
    }

    return new Vec3(
        Math.Clamp(color.X, 0.0, 1.0),
        Math.Clamp(color.Y, 0.0, 1.0),
        Math.Clamp(color.Z, 0.0, 1.0));
}
```

Quando o raio não atinge nenhum objeto, o método devolve a cor de fundo (0,02, 0,02, 0,05) — um tom escuro azulado que simula um céu noturno. Essa escolha é arbitrária e poderia ser substituída por um gradiente ou por uma imagem de ambiente sem alterar a estrutura do algoritmo.

A reflexão recursiva é o único mecanismo de iluminação global presente no sistema. Ela captura reflexos entre superfícies, mas não iluminação indireta difusa (o *color bleeding* característico de cenas reais). Para isso seria necessário *path tracing* ou radiosidade, ambos fora do escopo.

3.8. LOOP DE RENDERIZAÇÃO

O loop de renderização é implementado pelo método Render da classe Renderer. Ele percorre todos os pixels da imagem, gera o(s) raio(s) correspondente(s), rastreia-os e acumula a cor resultante em um *buffer* bidimensional de Vec3. O *buffer* é retornado ao chamador para conversão em imagem (Seção 3.9).

A implementação usa Parallel.For sobre as linhas da imagem. Como cada pixel é independente dos demais — não há estado compartilhado entre pixels —, a paralelização é trivial e produz *speedup* quase linear com o número de núcleos disponíveis. O particionamento por linha (e não por pixel) reduz o custo de sincronização, pois cada iteração externa processa uma linha inteira.

Para cada pixel, o método suporta **sobreamostragem** (samplesPerPixel):

- **Quando samplesPerPixel > 1:** lança-se múltiplos raios com *jitter* aleatório dentro do pixel e calcula-se a média das cores. Esse procedimento implementa o anti-*aliasing* estocástico descrito na Seção 3.2, suavizando as bordas dos objetos.
- **Quando samplesPerPixel = 1:** usa-se jx = jy = 0.5, ou seja, o centro do pixel — modo determinístico e mais rápido, útil durante ajustes de câmera.

O gerador de números aleatórios é semeado com 42 para reprodutibilidade: execuções sucessivas com os mesmos parâmetros produzem exatamente a mesma imagem, o que facilita a comparação e a depuração.

```
public Vec3[,] Render(int samplesPerPixel = 1)
{
    int w = Camera.Width;
    int h = Camera.Height;
    var buffer = new Vec3[h, w];
    var rnd = new Random(42);

    Parallel.For(0, h, j =>
    {
        for (int i = 0; i < w; i++)
        {
            Vec3 accum = Vec3.Zero;
            for (int s = 0; s < samplesPerPixel; s++)
            {
                double jx = samplesPerPixel == 1 ? 0.5 : rnd.NextDouble();
                double jy = samplesPerPixel == 1 ? 0.5 : rnd.NextDouble();

                Ray ray = Camera.GetRay(i, j, jx, jy);
                accum += Trace(ray, 0);
            }
            buffer[j, i] = accum / samplesPerPixel;
        }
    });

    return buffer;
}
```

O custo computacional é $O(W \cdot H \cdot S \cdot C)$, onde $W \cdot H$ é o número de pixels, S é samplesPerPixel e C é o número médio de raios por amostra (primário + sombras + reflexões). Esse custo é dominado por C, que por sua vez depende do número de luzes e da profundidade das reflexões. Para a cena de demonstração (Seção 6), com duas luzes e reflexões rasas, a renderização em 800 × 600 com S = 1 conclui em poucos segundos em hardware moderno.

O *buffer* retornado é uma matriz de Vec3 com dimensões [Height, Width], indexada como buffer[j, i], onde *j* é a linha (vertical) e *i* é a coluna (horizontal). Essa escolha de indexação é consistente com a convenção de coordenadas de imagem adotada na câmera (Seção 3.2), na qual *j* cresce de cima para baixo.

3.9. CONVERSÃO E EXPORTAÇÃO

A conversão do *buffer* de cores para uma imagem visualizável é implementada pela classe estática BitmapConverter. Ela recebe a matriz de Vec3 produzida pelo Renderer e devolve um Bitmap do .NET, pronto para exibição no controle PictureBox (Seção 5) ou gravação em disco.

A implementação usa Bitmap.LockBits em vez de Bitmap.SetPixel. A diferença é significativa:

- **SetPixel:** realiza uma chamada de interpolação por pixel, o que é proibitivamente lento para imagens de alta resolução;
- **LockBits:** expõe o *buffer* de memória subjacente da imagem como um ponteiro (ou como um *array* de bytes copiado via Marshal.Copy), permitindo escrever os pixels diretamente em um laço.

O formato escolhido é Format24bppRgb, no qual cada pixel ocupa três bytes na ordem **B, G, R** (azul, verde, vermelho) — ordem herdada de convenções históricas da plataforma Windows e diferente da ordem RGB usada no restante do código. A conversão de Vec3 (com valores em [0, 1]) para bytes (em [0, 255]) é feita por multiplicação por 255 e *clamp*, seguida de truncamento.

O uso de stride em vez de 3*w* é essencial: o .NET alinha cada linha da imagem em múltiplos de 4 bytes, de modo que Stride pode ser maior que 3*w*. Ignorar esse detalhe produziria imagens desalinhadas. O bloco try/finally garante que UnlockBits seja chamado mesmo em caso de exceção, evitando vazamento do *handle* da imagem.

A exportação em PNG é delegada ao método Save da classe Bitmap, que aceita um caminho de arquivo e um ImageFormat. O formato PNG foi escolhido por ser sem perdas e amplamente suportado, sem necessidade de bibliotecas externas. A interface gráfica (Seção 5) envolve essa chamada em um SaveFileDialog, oferecendo ao usuário a escolha do caminho e do nome do arquivo, com sugestão automática baseada em data e hora (raytrace_yyyyMMdd_HHmmss.png).

```
public static Bitmap ToBitmap(Vec3[,] buffer)
{
    int h = buffer.GetLength(0);
    int w = buffer.GetLength(1);
    var bmp = new Bitmap(w, h, PixelFormat.Format24bppRgb);

    BitmapData data = bmp.LockBits(
        new Rectangle(0, 0, w, h),
        ImageLockMode.WriteOnly,
        PixelFormat.Format24bppRgb);

    try
    {
        int stride = data.Stride;
        byte[] pixels = new byte[stride * h];

        for (int j = 0; j < h; j++)
        {
            int rowOffset = j * stride;
            for (int i = 0; i < w; i++)
            {
                Vec3 c = buffer[j, i];
                int r = (int)(Math.Clamp(c.X, 0.0, 1.0) * 255.0);
                int g = (int)(Math.Clamp(c.Y, 0.0, 1.0) * 255.0);
                int b = (int)(Math.Clamp(c.Z, 0.0, 1.0) * 255.0);

                int idx = rowOffset + i * 3;
                pixels[idx + 0] = (byte)b;
                pixels[idx + 1] = (byte)g;
                pixels[idx + 2] = (byte)r;
            }
        }

        Marshal.Copy(pixels, 0, data.Scan0, pixels.Length);
    }
    finally
    {
        bmp.UnlockBits(data);
    }

    return bmp;
}
```

4. CENÁRIO DE DEMONSTRAÇÃO

Esta seção descreve a cena utilizada para validar e demonstrar o sistema. A cena foi projetada para exercitar, simultaneamente, todos os componentes implementados: as duas primitivas geométricas (esfera e triângulo), os três termos do modelo de Phong, sombras projetadas, reflexão recursiva e múltiplas fontes de luz. A descrição é acompanhada da justificativa de cada escolha, explicitando a que conceito da Seção 2 cada elemento da cena se vincula.

4.1. DESCRIÇÃO DA CENA

A cena é montada pelo método estático BuildScene da classe Form1. Ela contém **cinco objetos** (duas esferas e três triângulos) e **duas fontes de luz pontuais**, além de uma luz ambiente global. A Tabela 1 resume os objetos; a Tabela 2, as luzes; e a Tabela 3, os materiais.

Objeto	Primitiva	Parâmetros geométricos	Material
Esfera vermelha	Sphere	centro (-1.5, 0.8, 0.0), raio 0.8	matRed
Esfera azul	Sphere	centro (1.5, 0.6, -1.0), raio 0.6	matBlue
Chão (triângulo 1)	Triangle	V0 = (-10, 0, -10), V1 = (10, 0, -10), V2 = (10, 0, 10)	matFloor
Chão (triângulo 2)	Triangle	V0 = (-10, 0, -10), V1 = (10, 0, 10), V2 = (-10, 0, 10)	matFloor
Triângulo vertical	Triangle	V0 = (0, 0, -2.5), V1 = (1.5, 0, -3.5), V2 = (-1.5, 0, -3.5)	matGreen

Os dois triângulos do chão formam, em conjunto, um quadrado de lado 20 no plano y = 0, com orientação para cima. A divisão em dois triângulos é necessária porque a primitiva implementada é o triângulo, não o quadrilátero; a diagonal comum é escolhida de modo a não introduzir descontinuidade visual perceptível. O triângulo

vertical, inclinado em relação ao plano do chão, serve como superfície de teste para o algoritmo de Möller–Trumbore em orientação arbitrária.

Luz	Posição	Intensidade RGB	Papel
Principal	(-4, 6, 4)	(0.90, 0.90, 0.90)	Iluminação direta; projeta sombras visíveis
Preenchimento	(5, 4, 2)	(0.40, 0.40, 0.50)	Suaviza sombras e revela faces opostas

A luz ambiente global (Scene.Ambient) é fixada em (0.05, 0.05, 0.05), valor baixo que garante que regiões não iluminadas diretamente não fiquem completamente pretas, sem comprometer o contraste das sombras.

Material	Cor RGB	k_a	k_d	k_s	n	k_r
matRed	(0.85, 0.10, 0.10)	0.1	0.7	0.6	80	0.2
matBlue	(0.10, 0.20, 0.90)	0.1	0.7	0.5	120	0.0
matFloor	(0.60, 0.60, 0.60)	0.1	0.7	0.1	10	0.0
matGreen	(0.10, 0.80, 0.30)	0.1	0.7	0.4	40	0.0

A câmera inicial é posicionada em e = (0, 1.5, 6), com alvo em (0, 0.5, 0) e vetor *up* (0, 1, 0). O campo de visão vertical inicial é 60°, e a resolução padrão é 800 x 600. Todos esses parâmetros podem ser alterados pela interface (Seção 5), que recompõe a cena e re-renderiza a cada solicitação.

A disposição espacial é a seguinte: a câmera olha da frente para o fundo, ligeiramente acima do chão; a esfera vermelha está à esquerda e mais próxima; a esfera azul está à direita e mais ao fundo; o triângulo vertical fica entre as duas esferas, ao fundo, formando um plano inclinado que recebe sombra de ambas. O chão estende-se por 20 unidades em cada direção, garantindo que os raios que não atingem os objetos principais encontrem uma superfície visível em vez de escapar para o fundo.

4.2. JUSTIFICATIVA DAS ESCOLHAS

Cada elemento da cena foi escolhido para exercitar um aspecto específico do sistema. A análise a seguir percorre as decisões de projeto.

Presença simultânea de esferas e triângulos: As duas primitivas implementadas são geometricamente distintas e exercitam algoritmos diferentes: a esfera resolve uma equação quadrática com raízes simétricas em torno do centro; o triângulo resolve um sistema baricêntrico com testes de pertinência em u, v e $u + v$. Incluir ambas na mesma cena permite verificar que o Scene.Trace integra corretamente os dois tipos de interseção e escolhe o hit mais próximo independentemente da primitiva. Além disso, o triângulo é a primitiva fundamental de malhas; sua presença antecipa a extensão natural do sistema para modelos complexos.

Esferas de raios e posições distintos: A esfera vermelha (raio 0.8, mais próxima) e a azul (raio 0.6, mais ao fundo) produzem diferenças de escala e de profundidade que exercitam a projeção em perspectiva: a esfera mais próxima aparece maior, e a mais distante, menor, como esperado. As posições laterais opostas também verificam a simetria da base da câmera e a correta atribuição de raios aos pixels à esquerda e à direita do centro.

Chão como superfície de recepção de sombras: O chão é essencial para tornar visíveis as sombras projetadas pelas esferas e pelo triângulo vertical. Sem ele, os raios de sombra não encontrariam superfície onde se manifestar, e o efeito passaria despercebido. O material matFloor, com k_s baixo e n pequeno, produz um aspecto fosco que não compete visualmente com os objetos principais, mas evidencia o contraste entre regiões iluminadas e sombreadas.

Triângulo vertical em orientação arbitrária: O triângulo vertical é inclinado em relação ao plano do chão e à direção de visão, o que verifica o algoritmo de Möller–Trumbore em uma configuração não trivial. Se o triângulo estivesse no plano do chão, o teste seria semelhante ao dos dois triângulos do piso; incliná-lo força o algoritmo a lidar com um determinante não unitário e com coordenadas baricêntricas em uma base oblíqua. A cor verde, distinta das demais, facilita a identificação visual da superfície e a verificação de que o sombreamento está sendo aplicado corretamente em ambos os lados (material *double-sided*).

Duas fontes de luz: A luz principal, à esquerda e acima, produz sombras alongadas para a direita, evidenciando a direção da incidência. A luz de preenchimento, à direita e mais fraca, suaviza as sombras e ilumina as faces que a luz principal não alcança. A combinação das duas verifica o laço do Shader sobre múltiplas luzes e o acúmulo de contribuições. Se houvesse apenas uma luz, o efeito da soma de termos difusos e especulares de fontes distintas não seria observável.

Materiais com coeficientes contrastantes: Os quatro materiais foram escolhidos para cobrir o espaço de parâmetros do modelo de Phong:

- matRed, com $k_s = 0.6$ e $n = 80$, produz um *highlight* estreito e intenso, típico de plástico polido. A refletividade $k_r = 0.2$ adiciona reflexão discreta, suficiente para que o chão e a esfera azul apareçam levemente refletidos na superfície vermelha, sem dominar a aparência.
- matBlue, com $k_s = 0.5$ e $n = 120$, produz um *highlight* ainda mais concentrado, verificando o efeito do expoente n sobre a largura do lobo especular.
- matFloor, com $k_s = 0.1$ e $n = 10$, é quase puramente difuso, adequado ao papel de superfície de fundo.
- matGreen, com $k_s = 0.4$ e $n = 40$, ocupa uma posição intermediária, exibindo *highlight* de largura média.

A variação sistemática de k_s e n entre os materiais permite observar, em uma única imagem, o espectro de comportamentos do modelo especular — do fosco ao polido.

Cores distintas para os objetos: As cores vermelha, azul, verde e cinza foram escolhidas para que cada objeto seja imediatamente identificável na imagem. Essa identificabilidade é útil não apenas esteticamente, mas como ferramenta de verificação: se a esfera azul aparece com tom avermelhado em alguma região, isso indica erro na composição de cores (por exemplo, troca de canais na conversão para Bitmap); se o triângulo verde aparece cinza, isso indica que o material não está sendo aplicado corretamente.

Câmera posicionada acima do chão e voltada para o centro: A posição (0, 1.5, 6), com alvo em (0, 0.5, 0), coloca a câmera ligeiramente acima do plano do chão, olhando para baixo em direção ao centro da cena. Essa configuração enquadra todos os objetos no campo de visão inicial de 60° e produz uma composição em perspectiva natural, na qual o chão recua em direção ao fundo. A altura 1.5 é suficiente para que

o chão ocupe a metade inferior da imagem sem cobrir as esferas; o alvo em 0.5 centraliza verticalmente os objetos principais.

Luz ambiente global baixa: O valor (0.05, 0.05, 0.05) foi escolhido para evitar que regiões em sombra fiquem totalmente negras. Em cenas reais, a luz indireta garante que sombras nunca sejam absolutamente escuras; como o sistema não implementa iluminação global difusa (apenas reflexão especular), a luz ambiente cumpre parcialmente esse papel. O valor é baixo o suficiente para que o contraste entre luz e sombra permaneça evidente, mas alto o suficiente para que detalhes em regiões sombreadas sejam perceptíveis.

Fundo escuro azulado: A cor de fundo (0.02, 0.02, 0.05), retornada pelo Renderer quando um raio não atinge nenhum objeto, simula um céu noturno e cria contraste com os objetos iluminados. Um fundo branco ou cinza claro reduziria o contraste e dificultaria a avaliação das sombras e dos *highlights*.

Em conjunto, essas escolhas produzem uma cena que é simultaneamente **demonstrativa** — cada elemento tem uma função de verificação — e **visualmente legível** — o observador identifica prontamente os objetos, as sombras, os *highlights* e os reflexos, podendo relacionar cada característica visual ao componente do sistema que a produziu. A cena não busca fotorrealismo; busca tornar visível, de forma controlada, o funcionamento do motor.

5. CONCLUSÃO

Este trabalho teve como objetivo implementar, do zero, um sistema de renderização por *ray tracing* que integrasse os principais conceitos de Computação Gráfica estudados na disciplina, disponibilizando uma interface gráfica capaz de configurar a cena, ajustar parâmetros de visualização e exportar as imagens produzidas. A análise do percurso realizado permite avaliar o que foi alcançado, os aprendizados consolidados e os caminhos que se abrem a partir do sistema construído.

O sistema implementado cobre o ciclo completo da síntese de imagens por *ray tracing*. A **álgebra vetorial** (Vec3) fornece a base sobre a qual todos os demais componentes se apoiam, com a convenção de direções unitárias que confere significado geométrico ao parâmetro *t* dos raios. O **gerador de raios** (Camera) traduz a geometria da câmera *pinhole* em raios primários, com base ortonormal, plano de imagem e suporte a *jitter* para anti-*aliasing*. As **interseções** raio-esfera e raio-triângulo (Sphere, Triangle) implementam, respectivamente, a equação quadrática na forma *half-b* e o algoritmo de Möller–Trumbore, ambos verificados com exemplos numéricos calculados manualmente. O **Modelo de Refletância de Phong** (Shader) combina os termos ambiente, difuso e especular, com tratamento correto dos casos em que a luz está atrás da superfície ou o observador está fora do lobo especular. As **sombras** (Scene.InShadow) são produzidas por raios de sombra com *offset* numérico, e a **reflexão recursiva** (Renderer.Trace) captura iluminação global entre superfícies reflexivas, com controle de profundidade. O **loop de renderização** (Renderer.Render) paraleliza o processamento por linha e acumula o resultado em um *buffer* de cores, e a **conversão e exportação** (BitmapConverter) traduz esse *buffer* em imagem, com gravação em PNG.

A implementação de um *ray tracer* do zero, ainda que em escala reduzida, oferece uma perspectiva sobre a Computação Gráfica que o uso de bibliotecas prontas não proporciona. Cada conceito — a projeção em perspectiva, a equação quadrática da esfera, o sistema baricêntrico do triângulo, a soma dos termos de Phong, o *offset* dos raios de sombra — deixa de ser uma fórmula a memorizar e passa a ser uma decisão de projeto cujas consequências são visíveis na imagem produzida. O sistema construído é, nesse sentido, tanto um artefato técnico quanto um instrumento de aprendizado: ele funciona, produz imagens coerentes com a teoria e

permite que o observador relacione cada característica visual ao componente que a gerou.

A integração com Windows Forms acrescenta uma dimensão prática: o sistema não é apenas um exercício de renderização, mas uma aplicação utilizável, na qual o usuário ajusta parâmetros, observa o resultado e exporta imagens. Essa dimensão reforça o caráter completo do trabalho, que percorre o ciclo da Computação Gráfica aplicada — da modelagem à síntese, da teoria à interface.